AI COMPUTING
CONTINUUM



The Autonomous AI-RU

Closing the RIC intelligence gap at the radio through the ruApp

Laawanya Kishor *Director & Founder, UGECO PRIVATE LIMITED*

Rexy Geevarghese *Principal RAN Architect, Bell Canada*

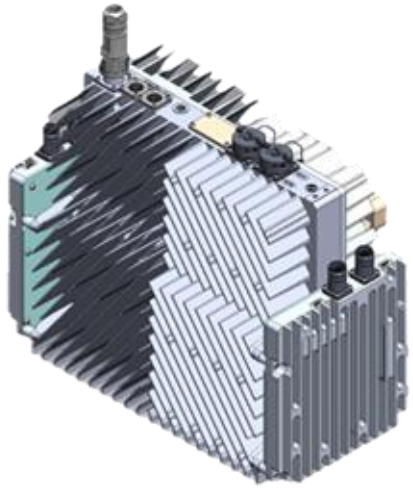
Autonomous AI Wireless Access Journey

Pivoting from lowest-CapEx-at-all-costs RUs (LCaAC) and declining ARPU to **Autonomous Radio Units (Au-RUs)**

What changes

Local intelligence and AI/ML embedded in the radio unit itself not concentrated solely in the baseband or GPU-centric layers.

The RU moves from a pure CapEx cost centre to an intelligent driver that lowers total cost of ownership.



Radio unit: where the intelligence now lives

New use cases

- Reduced control-loop latency for local decision-making the ruApp
- Manufacturing cost reductions through radio test automation
- New revenue streams, e.g. 6G integrated sensing and communication (ISAC)

The Au-RU thesis: intelligence belongs where the radio's moments happen in the RU, governed from above.

The RIC can decide. The radio can act. Between them lies an open gap.

Intelligence reaches the DU, not the radio

RIC loops run at 10 ms–1 s (E2) and >1 s (A1/O1). The radio's moments symbol, slot, PRACH last microseconds. Intent arrives after the moment has passed.

The blocker is time, not topology

Wake-on-demand, micro-sleep and per-PAM control need decisions in µs–ms. The fastest RIC loop is 10 ms; the management-plane path costs seconds , four orders of magnitude no transport fix can close.

A local ruApp closes the gap

Intelligence resident in the RU decides on the spot, inside a policy envelope the RIC still governs. No round trip on the critical path the latency of intelligence transfer stops mattering.

Actuated over the standard OCUDU → Open M-Plane loop, closed by PM ingestion. The radio becomes a supervised decision-maker on the same open management loop.

Autonomous AI-RU Prototype

OCUDU
O-CU · O-DU
M-Plane NETCONF client

Open M-Plane

eCPRI

Autonomous Radio Unit (Au-RU)

Au-RU management agent(s)

AI ruApp(s) · local decisions

K8s worker node

M-Plane CNF · NETCONF/YANG

FH-Plane CNF · eCPRI C/U-plane

Radio hardware interface

Firmware

RF front end | PHY | DFE

Dev environment

Lattice–NVIDIA Holoscan Sensor Bridge



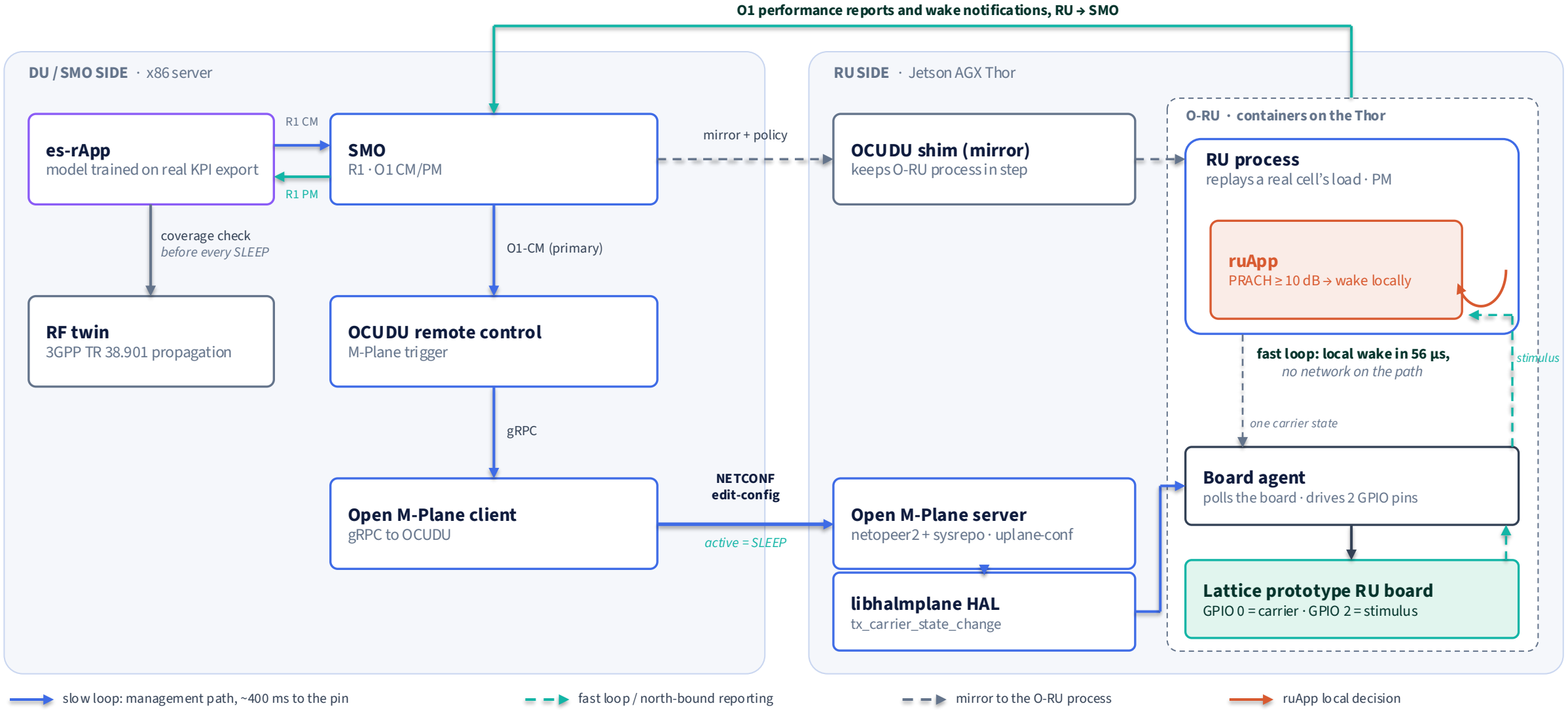
Lattice CertusPro-NX · sensor-to-Ethernet bridge



NVIDIA AGX Thor · 40×10G breakout

The ruApp is a first-class workload inside the RU containerized beside the M-Plane and fronthaul CNFs, directly above the radio hardware.

Lab architecture: two loops, one open management path



It ran: real KPI data in, real hardware out

Lab run, 18 Sep 2026. *Model trained on a real operator KPI export (29 cells, 8 days). The O-RU replayed one real cell on one held-out night; the lab's Lattice prototype RU board sat at the end of the management path.*

400 ms
rApp SLEEP → board pin LOW
the whole standard management path, server to server to FPGA pin

56 µs
ruApp wake decision
local, triggered from the board when real demand came back

29 ms
stimulus seen → carrier HIGH
host software over Ethernet around the board; not the decision

0
windows slept through demand
the rApp alone would have slept through 15 and 166

The night, as it replayed

00:32 SLEEP reaches the board
1.82 % PRB seen; model 0.99 sure of low load; RF twin allows it; 400 ms later GPIO reads 0

01:22 Real demand returns
replayed load crosses the learned 4.66 % threshold; doorbell pin raised on the board

+56 µs ruApp decides locally
the policy envelope that travelled with SLEEP allows a wake on PRACH

+29 ms Carrier HIGH, burst served
SMO receives the wake notification; the carrier carries 8.6 % PRB in the next half hour

On the full real histories, the ruApp is what makes sleep safe

Same code, same policy, window by window over the real exports. *Energy priced with the lab's measured power curve (200 W idle → 400 W full load) and 60 W for a sleeping carrier.*

A · Depth: 3 sites, 30 days, last 5 held out 6.2 % mean PRB · 3,537 train / 720 validation windows

Variant	Energy saved	Slept through demand	What it means
es-rApp + ruApp	**15.6 %**	**0 windows**	13 local wakes gave every demand window back; 85 % of the oracle ceiling
rApp only, no local wake	**17.6 %**	15 windows	8 hours of real demand unserved in 5 days: the saving is not real
Oracle, perfect foresight	**18.4 %**	0 windows	the ceiling

B · Breadth: 29 cells, 8 days, last 2 held out 14.9 % mean PRB · 8,352 train / 2,784 validation windows

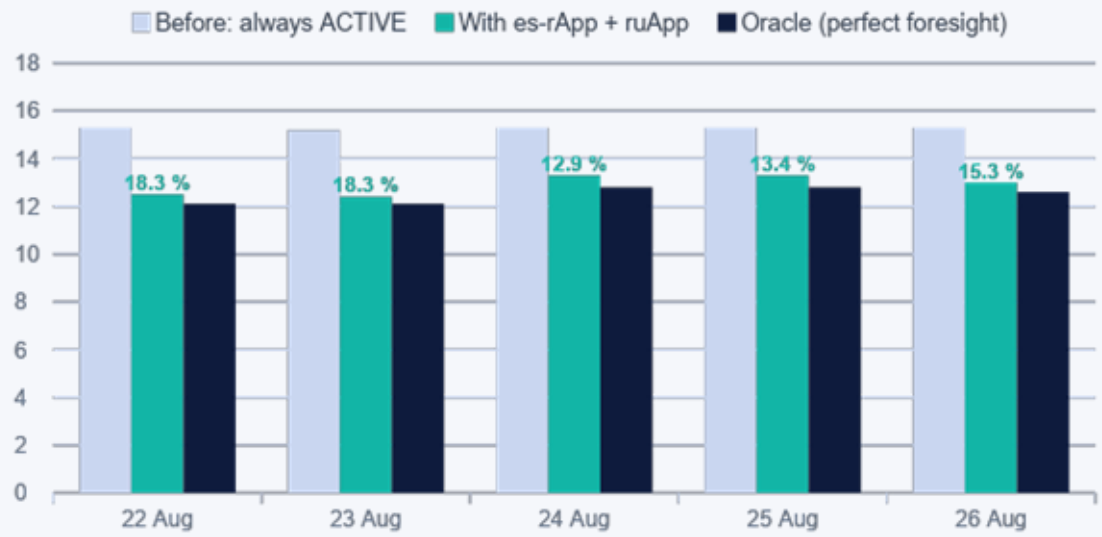
Variant	Energy saved	Slept through demand	What it means
es-rApp + ruApp	**12.1 %**	**0 windows**	73 local wakes; 69 % of the ceiling on a 2-day validation
rApp only, no local wake	**17.8 %**	166 windows	83 hours of real demand unserved in 2 days
Oracle, perfect foresight	**17.6 %**	0 windows	the ceiling

The rApp alone posts a higher number only because it sleeps through real demand. The ruApp gives back every one of those windows and keeps ~85 % of the saving.

Depth and breadth: the same policy over the real histories

A · Depth 3 sites · 30 days of history · last 5 days held out

kWh per day, all 3 cells · label = energy saved with the ruApp

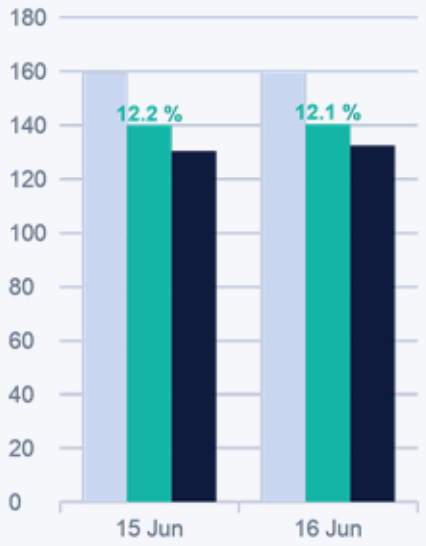


Site by site · energy saved over the 5 validation days



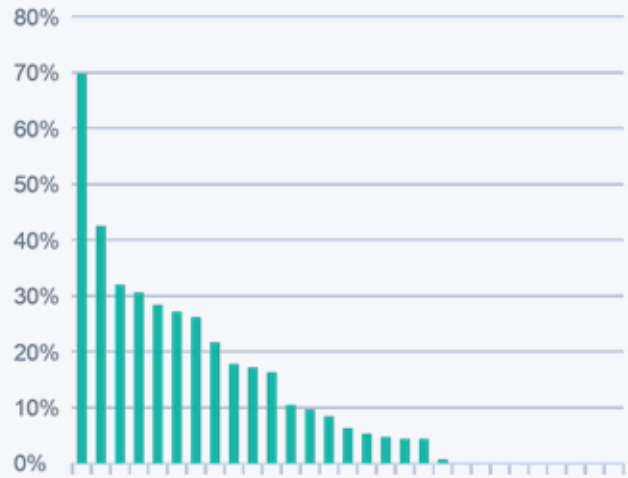
B · Breadth 29 cells · 8 days of history · last 2 days held out

kWh per day, all 29 cells · before with ruApp oracle



Every cell, not just the average

energy saved per cell, sorted · median 6.3 %



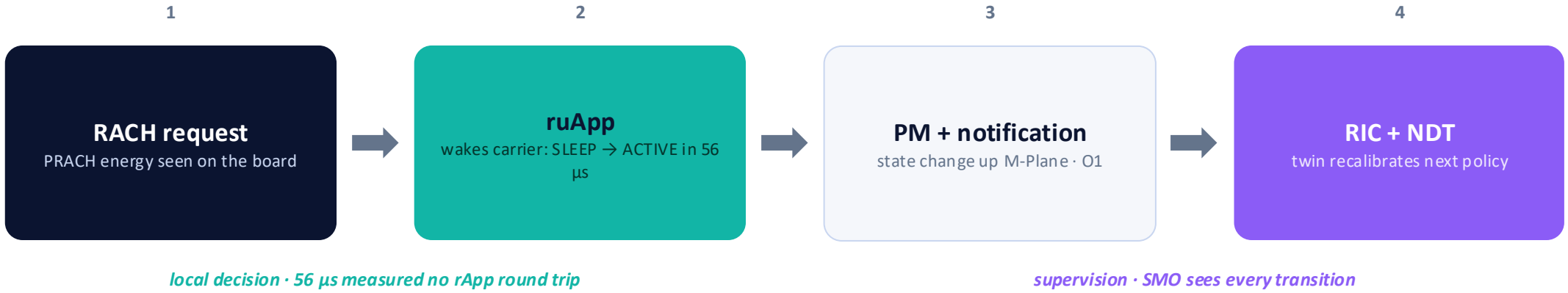
What the spread means

- 9 busy cells never drop below the threshold and never sleep: the policy costs them nothing.
- Two near-empty sectors sleep almost all day (69.8 % and 42.5 %).
- The 12.1 % estate figure is carried by the quiet half; median cell saves 6.3 %.

Priced with the lab power curve (200 W idle → 400 W full load) and 60 W for a sleeping carrier. The oracle knows in advance which windows are quiet; the gap to it is the honest measure of the forecast.

The moment that proves the ruApp

Setup: *the rApp has already set the carrier to SLEEP via O1 → OCUDU → M-Plane: the slow loop, 400 ms to the board. Then real demand returns.*



Why this cannot be an rApp: the management path measured 400 ms end to end; the ruApp decided in 56 µs. Four orders of magnitude, on the same hardware, in the same run.

One run, both loops: slow-loop policy put the carrier to sleep · fast-loop autonomy woke it when real demand returned · PM taught the twin.

Three phases to an open Autonomous AI-RU

PHASE 1 · DONE

The actuation path

Policy flowed SMO → O-RU on standard NETCONF/YANG, ending on a real FPGA pin. No custom interface, no vendor path.

PHASE 2 · DONE, 18 SEP 2026

Close both loops

rApp set the envelope on real KPI data; the ruApp woke the carrier from the board in 56 µs when real demand returned. Zero missed windows.

PHASE 3 · NEXT

Standardize

Publish the policy → M-Plane → ruApp contract as a community pattern; measure sleep power on real radio hardware; bring a reference RU under it.

Let's make the radio intelligent: Call to Action

1. Run the demo: github.com/opencomputeproject/ocp-auRu
    - Clone the stack: OCUDU, Open M-Plane, mock AI-RU and reproduce the wake-on-demand loop yourself.
2. Contribute a ruApp *or other AI/ML designs to the Au-RU*
    - Bring sleep models, wake policies, per-PAM strategies, local intelligence that runs inside the policy envelope.
3. Shape the pattern
    - Co-author the policy → M-Plane → ruApp contract with us, and run it on your radio hardware: the one reading we still need is measured sleep power.

Thank You!

